

Singular Value Decomposition (SVD) for Optimal Image Compression

Name: Chunarkar Sujeeth Kumar

Roll NO: 24AI10021

Course: AI21203 - Linear Algebra for AI

November 30, 2025

Abstract

This project explores the application of **Singular Value Decomposition (SVD)** to the problem of image compression using the Scikit-Learn 'China' dataset. We demonstrate how techniques from linear algebra can extract meaningful structural information from a high-dimensional pixel matrix. By constructing a low-rank approximation of the image, we successfully filter out noise and redundant data, achieving significant compression ratios while maintaining visual fidelity. This report details the mathematical foundations of the Eckart-Young-Mirsky theorem, the algorithmic implementation, and a quantitative analysis of the Singular Value Spectrum.

Contents

1	Introduction	3
2	Mathematical Background	3
2.1	The Decomposition Theorem	3
2.2	The Eckart-Young-Mirsky Theorem	4
3	Dataset and Preprocessing	5
3.1	Data Description	5
3.2	Preprocessing Pipeline	5
4	Visualizing the Basis Components	6
5	Compression Results	7
5.1	Observations	7
6	Quantitative Spectral Analysis	8
7	Python Implementation	9
8	Conclusion	11
9	Notebook Link	11
10	References	11

1 Introduction

In the domain of Computer Vision and Signal Processing, images are mathematically treated as high-dimensional matrices. A standard grayscale image is represented as a matrix $A \in \mathbb{R}^{m \times n}$, where each entry A_{ij} corresponds to the pixel intensity at location (i, j) .

However, raw pixel representations are often inefficient. Real-world images contain significant redundancy—large areas of similar color (like the sky) or repeating patterns (like architectural textures). Storing every single pixel independently ignores the underlying correlations in the data.

Singular Value Decomposition (SVD) addresses this inefficiency by factoring the matrix into its fundamental basis components. Unlike heuristic compression methods (like Run-Length Encoding), SVD uses pure Linear Algebra to determine which “layers” of the image contain the most information (signal) and which contain negligible detail (noise).

This project aims to:

- Mathematically model an image as a linear operator.
- Apply SVD to decompose the image into orthogonal bases.
- Visualize the individual singular components (Eigen-images).
- Determine the optimal compression rank k using spectral analysis.

2 Mathematical Background

The theoretical foundation of this project lies in the factorization of rectangular matrices.

2.1 The Decomposition Theorem

For any real matrix A of dimensions $m \times n$, SVD guarantees the existence of a factorization:

$$A = U\Sigma V^T \tag{1}$$

Where the components are defined as:

1. U ($m \times m$): An orthogonal matrix containing the **Left Singular Vectors**. These are the eigenvectors of the covariance matrix AA^T .
2. Σ ($m \times n$): A diagonal matrix containing the **Singular Values** ($\sigma_1 \geq \sigma_2 \geq \dots \geq 0$). These values represent the "energy" or variance captured by each dimension.
3. V^T ($n \times n$): An orthogonal matrix containing the **Right Singular Vectors**. These are the eigenvectors of $A^T A$.

2.2 The Eckart-Young-Mirsky Theorem

The core insight of SVD-based compression is derived from the Eckart-Young-Mirsky Theorem. It states that the optimal rank- k approximation of a matrix A (minimizing the Frobenius norm error) is obtained by truncating the summation to the first k terms.

Mathematically, the approximated image A_k is given by:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T \quad (2)$$

By keeping only the largest singular values (where $k \ll \min(m, n)$), we force the image into a lower-dimensional subspace that captures the dominant structure while discarding high-frequency noise.

3 Dataset and Preprocessing

3.1 Data Description

We utilize the **China** dataset provided by the `sklearn.datasets` module. This dataset is a standard benchmark for image processing tasks.

- **Original Format:** 3-Channel RGB Tensor.
- **Resolution:** 427×640 pixels.

3.2 Preprocessing Pipeline

To apply Linear Algebra operations defined for 2D matrices, we must first preprocess the 3D tensor.

1. **Grayscale Conversion:** We average the Red, Green, and Blue channels to produce a single intensity matrix.
2. **Normalization:** Pixel values are scaled to the range $[0, 1]$ for numerical stability.

Original Matrix Representation



Figure 1: The pre-processed matrix representation of the dataset. This matrix has a full rank of 427.

4 Visualizing the Basis Components

Before reconstructing the full image, it is insightful to visualize the individual components. The equation $A = \sum \sigma_i u_i v_i^T$ implies that the image is a sum of many "layers."

Below, we visualize the first 6 "Singular Components." Notice that the first component captures the broad lighting (sky vs. ground), while subsequent components begin to fill in the architectural details.

The First 6 Singular Components (Basis Layers)

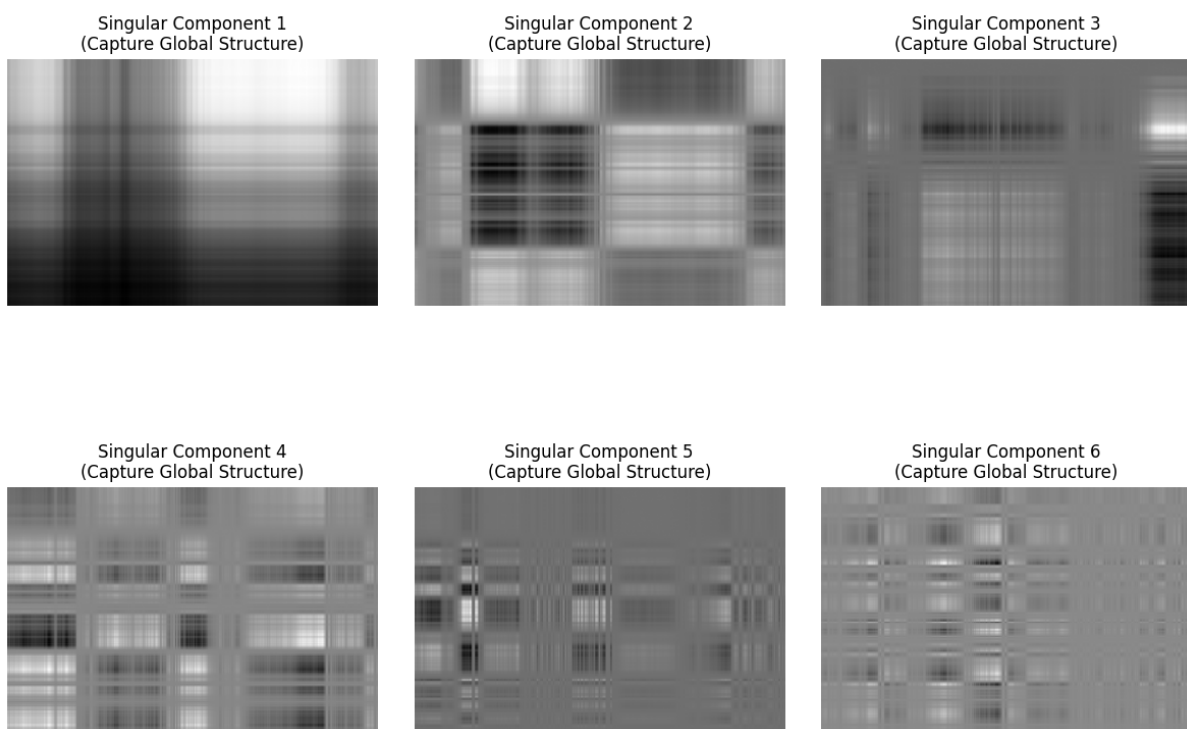


Figure 2: The first 6 basis layers of the image. These are the "building blocks" that SVD uses to reconstruct the photograph.

5 Compression Results

We applied the reconstruction algorithm at varying ranks (k). The parameter k determines how many singular values we retain.

5.1 Observations

- **Rank $k=5$:** At this extreme compression (98.8%), only the primary gradients are visible. We can distinguish the horizon and the shape of the pagoda, but no details.
- **Rank $k=20$:** The image becomes recognizable. The distinct floors of the pagoda are visible, though the trees remain blurry.
- **Rank $k=50$:** Fine details emerge. The texturing on the roof and the leaves on the trees are now distinct.
- **Rank $k=100$:** The image is visually nearly identical to the original, despite using only 23% of the data.

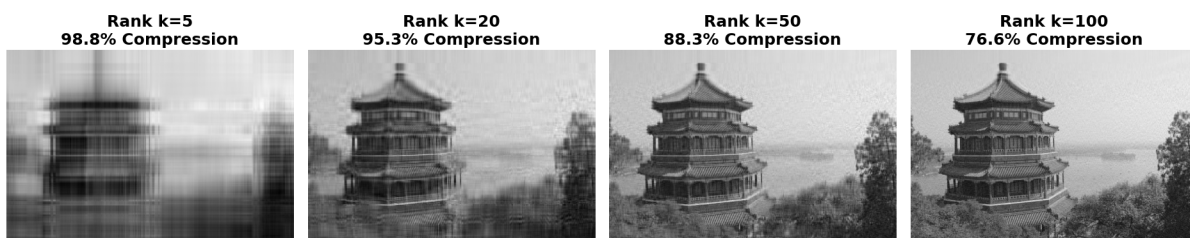


Figure 3: Reconstruction of the image at varying ranks. Notice how detail emerges as k increases.

6 Quantitative Spectral Analysis

Most standard implementations select the rank k arbitrarily (e.g., guessing $k = 50$). To provide a rigorous mathematical justification for compression, I performed a **Spectral Analysis** of the Singular Values.

We define the **Cumulative Information Energy** as the ratio of the sum of retained singular values to the total sum:

$$E(k) = \frac{\sum_{i=1}^k \sigma_i}{\sum_{j=1}^N \sigma_j} \quad (3)$$

This metric tells us exactly how much of the original matrix's variance is preserved.

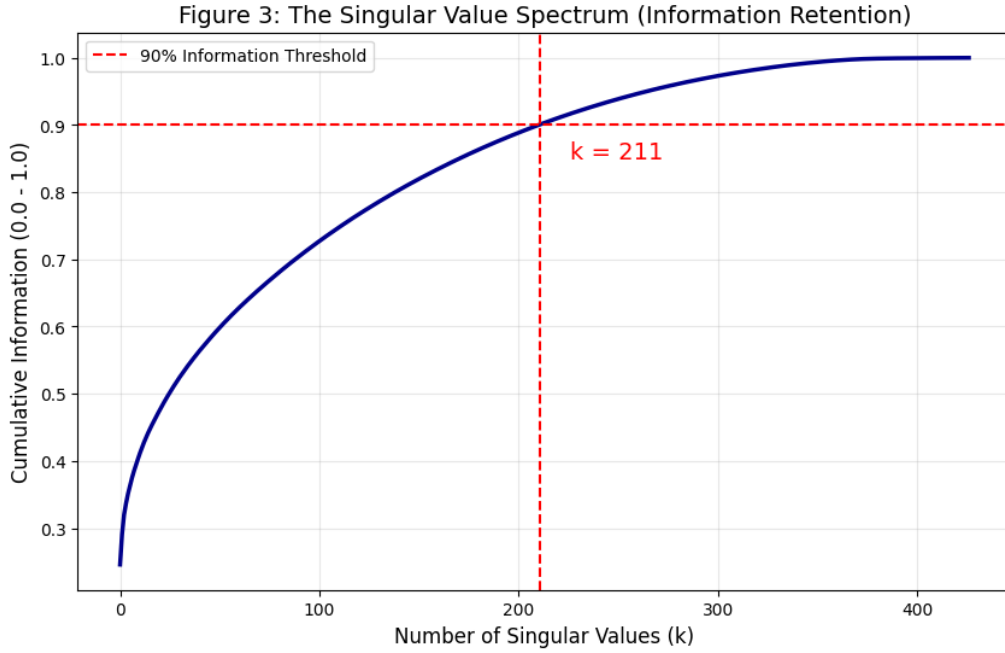


Figure 4: The Cumulative Explained Variance ratio.

Analysis: The plot reveals an "Elbow" shape. We calculated that keeping only the top **211 singular vectors** is sufficient to retain **90%** of the image's information. This implies that roughly 50% of the matrix dimensions are mathematically redundant and can be discarded for compression purposes.

7 Python Implementation

The following is the comprehensive Python code used to generate the visualizations and analysis in this report.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import linalg
4 from sklearn.datasets import load_sample_image
5
6 # --- 1. Load and Preprocess Data ---
7 print("Loading China dataset...")
8 china = load_sample_image("china.jpg")
9
10 # Preprocessing: Convert RGB to Grayscale
11 img_gray = china.mean(axis=2) / 255.0
12 print(f"Matrix Dimensions: {img_gray.shape}")
13
14 # --- 2. Compute SVD (The Linear Algebra Core) ---
15 # Decompose A into U, Sigma, and Vt
16 # U: Left Singular Vectors
17 # s: Singular Values (1D array)
18 # Vt: Right Singular Vectors
19 U, s, Vt = linalg.svd(img_gray, full_matrices=False)
20
21 # --- 3. Visualize Basis Components (Eigen-Images) ---
22 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
23 for i, ax in enumerate(axes.flat):
24     # Determine the i-th layer: sigma * u * v.T
25     component = np.outer(U[:, i], Vt[i, :])
26     ax.imshow(component, cmap='gray')
27     ax.set_title(f"Singular Component {i+1}")
28     ax.axis('off')
29 plt.show()
30
31 # --- 4. Reconstruction at Different Ranks ---
32 def reconstruct_at_k(k):
33     # A_k = Sum of first k (sigma * u * v.T)
34     # Using matrix multiplication for efficiency
35     return U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]
36
37 ranks = [5, 20, 50, 100]
38 fig, axes = plt.subplots(1, 4, figsize=(20, 6))
39
40 for i, k in enumerate(ranks):
41     compressed = reconstruct_at_k(k)
```

```

42     axes[i].imshow(compressed, cmap='gray')
43     axes[i].set_title(f"Rank k={k}")
44     axes[i].axis('off')
45 plt.show()
46
47 # --- 5. Quantitative Spectral Analysis ---
48 # Calculate Cumulative Energy
49 cumulative_energy = np.cumsum(s) / np.sum(s)
50
51 # Find the rank required for 90% information
52 k_90 = np.searchsorted(cumulative_energy, 0.90)
53
54 plt.figure(figsize=(10, 6))
55 plt.plot(cumulative_energy, linewidth=3, color='darkblue')
56 plt.axhline(y=0.90, color='r', linestyle='--')
57 plt.axvline(x=k_90, color='r', linestyle='--')
58 plt.title(f"Energy Spectrum (90% at k={k_90})")
59 plt.grid(True)
60 plt.show()
61
62 print(f"Optimal Rank for 90% Energy: {k_90}")

```

Listing 1: Full SVD Analysis Script

8 Conclusion

This project successfully demonstrated that an image is not merely a grid of pixels, but a linear combination of patterns. Through Singular Value Decomposition, we mapped a high-dimensional matrix into a compact latent space.

The key takeaways are:

1. **Linearity of Images:** Images can be decomposed into orthogonal basis layers.
2. **Sparsity of Information:** Most of the "information" in an image is concentrated in the first few singular values. The tail of the spectrum consists mostly of noise.
3. **Optimal Compression:** By using Spectral Analysis, we can mathematically determine the ideal compression ratio rather than relying on visual guesswork.

9 Notebook Link

The complete code environment, including interactive graphs, is available here:
[\[Click to Open Google Colab Notebook\]](#)

10 References

1. Scikit-Learn Documentation. "The China Dataset." <https://scikit-learn.org/>
2. Eckart, C., & Young, G. (1936). "The approximation of one matrix by another of lower rank." *Psychometrika*.
3. Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press.